

Bài tập lớn – Level 2

Lập trình Hướng Đối tượng (Object-Oriented Programming)



Viện Trí tuệ Nhân tạo, Trường Đại học Công nghệ, ĐHQGHN

Ngày 19 tháng 4 năm 2026

Mục lục

I	Giới thiệu (Introduction)	3
II	Mục tiêu học tập (Learning Objectives)	4
II.1	Thực hành lập trình hướng đối tượng	4
II.2	Áp dụng nguyên lý trừu tượng hoá dữ liệu	4
II.3	Thực hành lập trình trò chơi với GUI	5
II.4	Làm quen với một số mẫu thiết kế cơ bản	5
III	Mã nguồn cung cấp (Starter Code)	7
III.1	Tư duy thiết kế (Design Mindset)	7
III.2	Cấu trúc chương trình (Program Structure)	11
III.3	Thành phần mã nguồn	13
IV	Hướng dẫn triển khai (Implementation Guide)	16
IV.1	Tiền triển khai	16
IV.2	Thiết kế luồng thực thi	19
IV.3	Hoàn thiện giao diện Terminal	21
IV.4	Hoàn thiện giao diện SDL	21

V Hướng dẫn thực nghiệm (Experiment Instructions)	23
V.1 Cài đặt môi trường	23
V.2 Kiểm tra cài đặt môi trường	24
V.3 Biên dịch chương trình	25
V.4 Chạy chương trình	28
V.5 Chạy mã chấm điểm (Grader)	29
V.6 Ghi nhận kết quả vào báo cáo	30
VI Yêu cầu báo cáo (Report Requirements)	32
VI.1 Định dạng và Quy định chung	32
VI.2 Cấu trúc nội dung (Outline)	32
VII Chính sách sử dụng công cụ AI (AI Usage Policy)	36
VII.1 Nguyên tắc chung	36
VII.2 Yêu cầu khai báo	36
VII.3 Những việc không được phép	36
VII.4 Mục tiêu của chính sách	37
VIII Nộp bài (Submission)	38
VIII.1 Các thành phần cần nộp	38
VIII.2 Cấu trúc thư mục	38
VIII.3 Định dạng nộp bài	39
VIII.4 Thời hạn nộp bài	39
VIII.5 Lưu ý	40

I. Giới thiệu (Introduction)

Sau khi đã hoàn thành **Level 1** với một phiên bản *TicTacToe* chạy trên *terminal*, sử dụng *logger* và áp dụng thuần túy *lập trình thủ tục (Procedural Programming)*, các bạn đã nắm được cấu trúc cơ bản của một chương trình trò chơi: từ *game loop*, xử lý sự kiện, đến kiểm tra điều kiện thắng.

Trong **Level 2**, bài tập sẽ được nâng cấp theo hướng **hiện đại và có cấu trúc hơn**, bằng cách chuyển sang *lập trình hướng đối tượng (Object-Oriented Programming – OOP)*. Thay vì tổ chức chương trình dưới dạng các hàm rời rạc, các bạn sẽ học cách mô hình hoá hệ thống thành các *lớp (classes)* và *đối tượng (objects)*, từ đó tăng khả năng mở rộng và bảo trì chương trình.

Đồng thời, các bạn sẽ làm quen với thư viện SDL (phiên bản 2 hoặc 3) để xây dựng *giao diện người dùng đồ họa (Graphical User Interface – GUI)*. Đây là bước chuyển quan trọng từ một chương trình chạy trên dòng lệnh sang một ứng dụng có tính trực quan, nơi người chơi có thể tương tác thông qua cửa sổ, chuột và bàn phím.

Mục tiêu chính của Level 2 bao gồm:

- **Thực hành lập trình hướng đối tượng:** tổ chức lại chương trình bằng các lớp dựa trên cấu trúc ban đầu (Engine, Interaction và Renderer)
- **Áp dụng nguyên lý trừu tượng hoá dữ liệu:** tách biệt rõ ràng giữa phần *logic* và phần *hiển thị*,
- **Thực hành lập trình trò chơi với GUI:** sử dụng SDL để hiển thị bàn cờ, xử lý sự kiện chuột/bàn phím và cập nhật giao diện,
- **Làm quen với một số mẫu thiết kế cơ bản:** như *Factory Pattern* để linh hoạt trong việc khởi tạo các thành phần của hệ thống.

Level này không chỉ giúp các bạn nâng cao kỹ năng lập trình, mà còn là bước đệm quan trọng để xây dựng các hệ thống phần mềm phức tạp hơn trong tương lai, nơi việc tổ chức mã nguồn và thiết kế kiến trúc đóng vai trò then chốt.

II. Mục tiêu học tập (Learning Objectives)

Bài tập này được thiết kế nhằm giúp các bạn củng cố và vận dụng các kiến thức đã học trong phần trừu tượng hoá dữ liệu và lập trình hướng đối tượng. Đồng thời thử sức với việc thiết kế một chương trình phần mềm với nhiều giao diện tương tác khác nhau. Sau khi hoàn thành bài tập, các bạn được kỳ vọng có thể đạt được các mục tiêu học tập sau.

II.1 Thực hành lập trình hướng đối tượng

Trong Level này, các bạn sẽ chuyển đổi chương trình từ phong cách *lập trình thủ tục* sang *lập trình hướng đối tượng* (Object-Oriented Programming - OOP). Việc tổ chức lại mã nguồn không chỉ dừng ở việc tách một file lớn thành nhiều file nhỏ, hay đơn thuần đóng gói các hàm vào trong class hoặc namespace. Quan trọng hơn, chương trình cần thể hiện rõ ràng **bốn nguyên lý cốt lõi của OOP**:

- **Đóng gói (Encapsulation):** che giấu dữ liệu và chỉ cho phép truy cập thông qua các giao diện được kiểm soát,
- **Kế thừa (Inheritance):** xây dựng các lớp mới dựa trên các lớp đã có, tái sử dụng và mở rộng hành vi,
- **Trừu tượng (Abstraction):** chỉ thể hiện những đặc điểm cần thiết, ẩn đi chi tiết cài đặt,
- **Đa hình (Polymorphism):** cho phép nhiều đối tượng khác nhau có thể được xử lý thông qua cùng một giao diện.

Chương trình của các bạn **bắt buộc phải thể hiện được cả bốn nguyên lý trên**. Trong phần báo cáo hoặc vấn đáp, các bạn cần chỉ rõ các thành phần cụ thể trong mã nguồn tương ứng với từng nguyên lý, thay vì chỉ nêu lý thuyết chung chung.

II.2 Áp dụng nguyên lý trừu tượng hoá dữ liệu

Nguyên lý *trừu tượng hoá dữ liệu* (Data Abstraction) được thể hiện rõ thông qua việc áp dụng **đóng gói** trong OOP. Người sử dụng một lớp không cần phải biết chi tiết dữ liệu được tổ chức hay xử lý như thế nào bên trong, mà vẫn có thể khởi tạo và sử dụng đối tượng thông qua các giao diện công khai.

Nhờ cơ chế *che giấu thông tin* (information hiding), các thành phần trong chương trình trở nên **độc lập hơn**, từ đó giúp việc lập trình, sửa lỗi và mở rộng hệ thống trở nên dễ dàng hơn. Đây cũng là một tiêu chí quan trọng trong việc đánh giá chất lượng mã nguồn, và sẽ được liên hệ chặt chẽ với các nguyên tắc thiết kế ở mục tiêu cuối cùng.

II.3 Thực hành lập trình trò chơi với GUI

Trong phần này, các bạn sẽ làm quen với thư viện đồ hoạ SDL (phiên bản 2 hoặc 3) để xây dựng *giao diện người dùng đồ hoạ* (*Graphical User Interface – GUI*). Mục tiêu chính là hoàn thiện phần *Renderer* và *Interaction*, giúp chương trình có khả năng hiển thị và tương tác trực quan.

Các bạn được **toàn quyền sáng tạo** trong cách sử dụng thư viện, miễn là đảm bảo chương trình hoạt động đúng chức năng. Một số hướng tiếp cận gợi ý:

- **Hiển thị:**

- Sử dụng các hàm vẽ cơ bản của SDL (vẽ đường thẳng, hình chữ nhật, hình tròn, ...),
- Sử dụng `SDL_Texture` để hiển thị hình ảnh,
- Sử dụng `SDL_ttf` để hiển thị chữ.

- **Tương tác:**

- Xử lý sự kiện bàn phím (`KEY`),
- Xử lý sự kiện chuột (`MOUSE`),
- (Nâng cao) hỗ trợ tay cầm (`JOYSTICK/CONTROLLER`).

Thông qua quá trình này, các bạn sẽ nhận ra sự khác biệt quan trọng giữa việc thiết kế một chương trình chạy trên *terminal* và một chương trình có *GUI*, đặc biệt là trong cách tổ chức *game loop*, xử lý sự kiện và cập nhật trạng thái hệ thống.

Có một điều cần được quan trọng lưu ý: đó là mặc dù được thực hành về giao diện đồ hoạ nhưng chương trình của các bạn vẫn phải thực thi được trên cả hai loại giao diện là dòng lệnh (*Terminal*) và đồ hoạ (*Graphic*). Chính vì vậy, việc chú trọng vào phần thiết kế chương trình sao cho mã nguồn có thể phối hợp tốt giữa cả 2 loại giao diện là một điểm cộng lớn.

II.4 Làm quen với một số mẫu thiết kế cơ bản

Để có thể áp dụng các *mẫu thiết kế* (*Design Patterns*), trước hết mã nguồn của các bạn cần tuân thủ tốt các nguyên lý của OOP. Mặc dù các bạn được cung cấp một bộ mã khởi đầu (*starter code*), việc thiết kế và cài đặt chi tiết vẫn mang tính sáng tạo và không bị gò bó.

Tuy nhiên, chương trình cần đảm bảo **không vi phạm các nguyên tắc SOLID**, bao gồm:

- **S – Single Responsibility Principle (SRP):** Mỗi lớp chỉ nên có một trách nhiệm chính → phụ thuộc thấp, gắn kết cao,
- **O – Open/Closed Principle (OCP):** Mở để mở rộng, đóng để sửa đổi → khuyến khích trừu tượng hoá và tách chiến lược,

- **L – Liskov Substitution Principle (LSP):** Lớp con phải có thể thay thế lớp cha mà không làm sai lệch hành vi,
- **I – Interface Segregation Principle (ISP):** Tránh ép buộc các lớp phải cài đặt những phương thức không cần thiết,
- **D – Dependency Inversion Principle (DIP):** Phụ thuộc vào trừu tượng thay vì cài đặt cụ thể.

Việc áp dụng các mẫu thiết kế sẽ được xem là **một điểm cộng**. Trong mã nguồn cung cấp, các bạn sẽ có sẵn một ví dụ về *Factory Pattern* để tham khảo và mở rộng.

Đối với bài tập này, việc hoàn thiện chức năng chỉ là một phần. Quan trọng hơn, các bạn cần **hiểu rõ thiết kế hiện tại của chương trình**, đồng thời có khả năng **đề xuất và cải tiến thiết kế** để đáp ứng tốt hơn các mục tiêu đã nêu. Đây là bước khởi đầu để các bạn làm quen với việc thiết kế các hệ thống phần mềm có quy mô *vừa đến lớn*, nơi kiến trúc và tư duy tổ chức mã nguồn đóng vai trò then chốt.

III. Mã nguồn cung cấp (Starter Code)

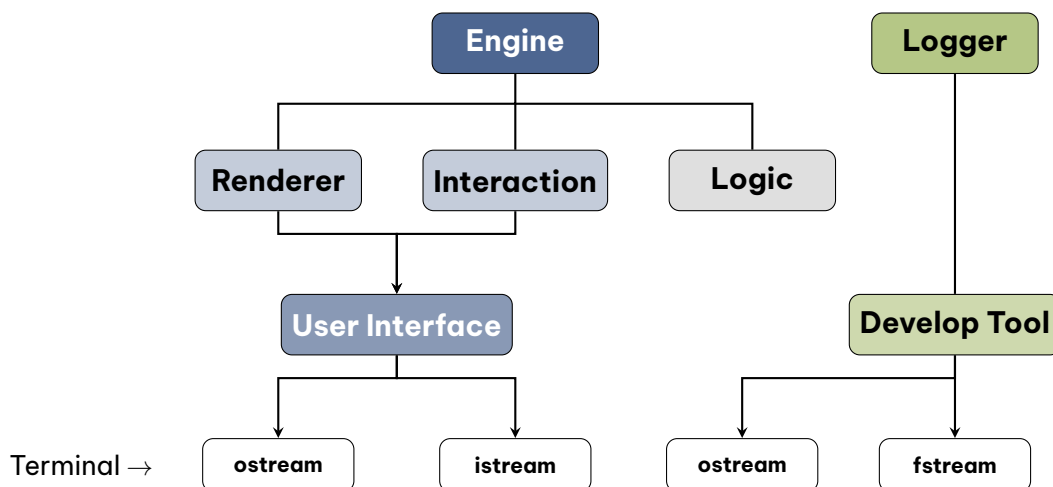
Để giúp các bạn tập trung vào việc cài đặt và thiết kế giao diện đồ họa của trò chơi, một thư mục mã nguồn **starter code** đã được cung cấp trong:

starter_code/

Phần này sẽ đưa ra cho các bạn Tư duy thiết kế một trò chơi với hai giao diện *Terminal* và *GUI*. Trước khi đi tới cấu trúc của chương trình, và phân tích các thành phần mã nguồn này.

III.1 Tư duy thiết kế (Design Mindset)

Nhắc lại từ Level 1, các bạn đã làm quen với một hệ thống trò chơi chạy trên *Terminal*, sử dụng `cin/cout` cho giao diện người dùng, và **Logger** (với các mức `DEBUG`, `INFO`, `WARN`, `ERROR`) để hỗ trợ quá trình phát triển. Ngay từ giai đoạn đó, tư duy **tách biệt các thành phần** trong chương trình đã bắt đầu được hình thành.



Hình 1: Kiến trúc chương trình level-1

Toàn bộ chương trình được điều phối bởi GameEngine (gọi tắt là Engine) trong hàm `main`. Engine bao gồm ba thành phần chính:

- **Renderer:** đảm nhiệm việc hiển thị giao diện (ở Level 1 là `cout`),
- **Interaction:** xử lý tương tác người dùng (ở Level 1 là `cin`),
- **Logic:** xử lý toàn bộ luật chơi và thuật toán kiểm tra trạng thái trò chơi.

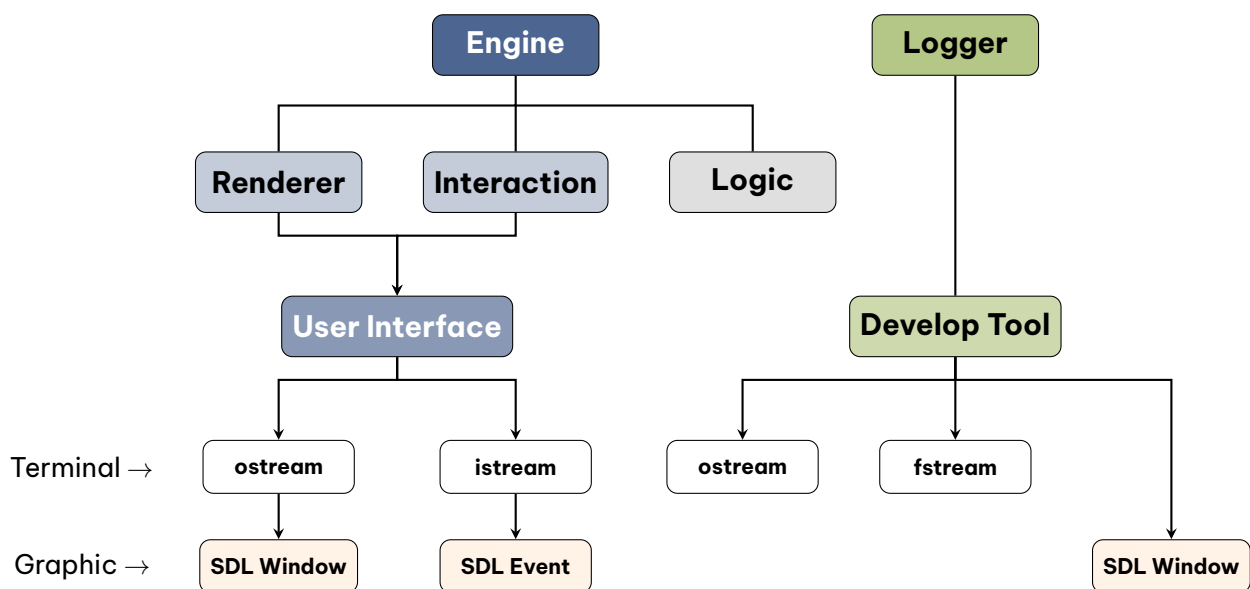
Bên cạnh đó, **Logger** được sử dụng như một công cụ hỗ trợ toàn cục (global), phục vụ cho việc theo dõi và debug chương trình. Mỗi thành phần trong hệ thống chỉ đảm nhận **một trách nhiệm duy nhất**, thể hiện rõ nguyên lý *Single Responsibility*.

UI vs Logger Một điểm quan trọng cần phân biệt rõ:

- **User Interface (UI):** dành cho **người chơi**, chỉ hiển thị những thông tin cần thiết để chơi game (ví dụ: “Hãy nhập kích thước bàn cờ”),
- **Logger:** dành cho **người phát triển**, cung cấp thông tin về trạng thái nội bộ của chương trình (ví dụ: “Đang khởi tạo bàn cờ với kích thước $n \times n$ ”).

Một sai lầm phổ biến là sử dụng Logger như UI (in ra cho người chơi xem). Điều này là **không đúng mục đích thiết kế**. UI và Logger phục vụ hai đối tượng hoàn toàn khác nhau.

Giao diện đồ họa (GUI) Khi mở rộng sang giao diện đồ họa (GUI), kiến trúc hệ thống có thể được hình dung như sau:



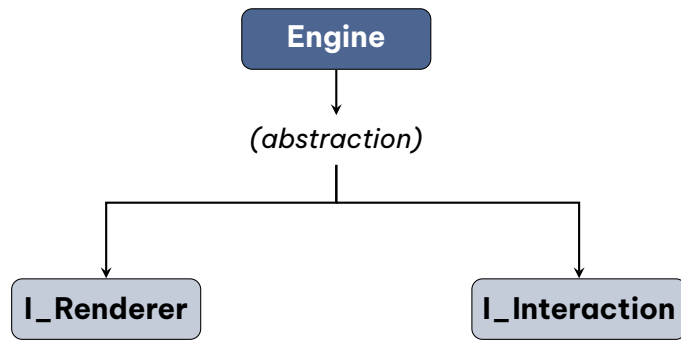
Hình 2: Kiến trúc chương trình level-2

Lúc này, **Logger hoàn toàn có thể hiển thị trên GUI** (ví dụ: trong trình duyệt có bật/tắt chế độ developer bằng phím F12), nhưng cần nhớ rằng nó **không phục vụ người chơi**, mà phục vụ người phát triển.

Trừu tượng hoá Giao diện Tư duy quan trọng tiếp theo nằm ở bên trong Engine. Engine **không cần biết** trò chơi đang chạy trên Terminal hay GUI. Nó chỉ cần biết rằng:

- Có một Renderer chịu trách nhiệm hiển thị,
- Có một Interaction chịu trách nhiệm nhận input,
- Logic là phần xử lý cốt lõi (không thay đổi).

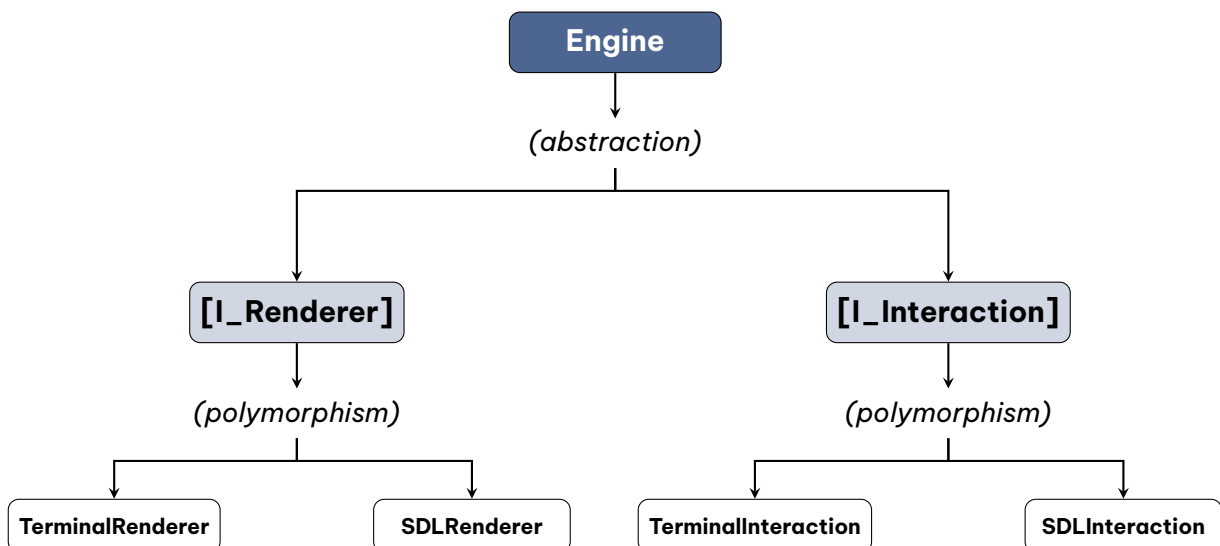
Điều này dẫn đến việc áp dụng **nguyên lý trừu tượng hoá** trong OOP:



Hình 3: Cấu trúc trừu tượng hóa của Engine

Trong đó, I_Renderer và I_Interaction là các *interface* (lớp trừu tượng), định nghĩa các hàm *virtual*. Chúng hoạt động như một *cổng giao tiếp*: ta biết có những chức năng nào, nhưng không cần biết chi tiết cài đặt bên trong.

Cụ thể hoá bằng Đa hình Từ đây, nguyên lý **đa hình (polymorphism)** được thể hiện rõ ràng. Với mỗi loại giao diện, ta có các lớp cài đặt cụ thể:



Hình 4: Cấu trúc đa hình trong thiết kế Engine

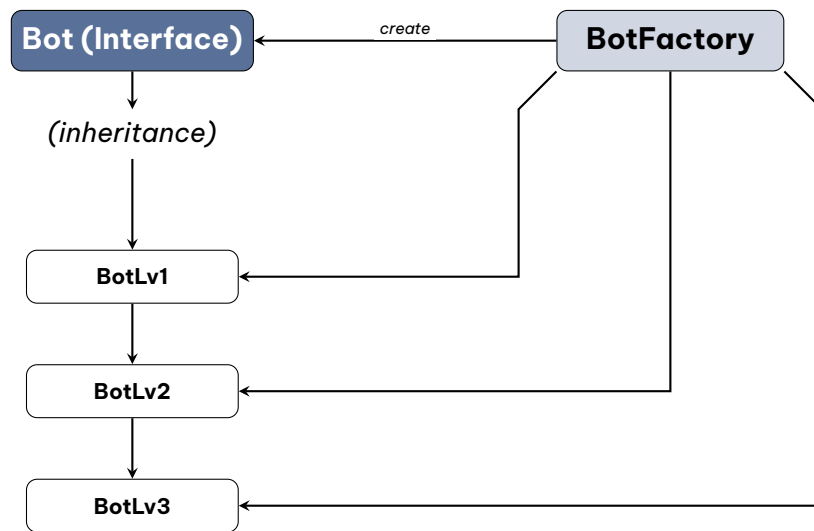
Như vậy:

- Engine chỉ làm việc với **interface** (trừu tượng),
- Các lớp cụ thể (Terminal, SDL) chỉ là phần **cài đặt**,
- Toàn bộ hệ thống trở nên **linh hoạt và dễ mở rộng**.

Nếu thiết kế đúng, chương trình có thể **chuyển đổi qua lại giữa Terminal và GUI** mà không cần thay đổi logic cốt lõi của Engine.

Mở rộng Bot với Kế thừa và Đa hình Bên cạnh đó, một tư duy khác kết hợp giữa **kế thừa (inheritance)** và **đa hình (polymorphism)** trong OOP. Một lớp cơ sở Bot (hoặc

IBot) sẽ định nghĩa giao diện chung cho tất cả các loại bot (ví dụ: hàm chọn nước đi). Từ đó, các lớp cụ thể như BotLv1, BotLv2, ... sẽ *kế thừa* từ lớp cơ sở và *cài đặt lại* hành vi theo từng mức độ chiến lược khác nhau.



Hình 5: Cấu trúc kế thừa và Factory Pattern cho hệ thống Bot

Thông qua cơ chế đa hình, Engine hoặc Logic có thể làm việc với con trỏ hoặc tham chiếu tới Bot mà không cần biết chính xác đó là loại bot nào. Việc khởi tạo bot cụ thể sẽ được tách riêng thông qua bot_factory, giúp hệ thống dễ dàng mở rộng (thêm bot mới mà không cần sửa code cũ). Đây là một ví dụ điển hình cho việc kết hợp **Inheritance + Polymorphism + Factory Pattern** trong thiết kế.

Đóng gói và tổ chức mã nguồn Tư duy thiết kế cuối cùng đưa toàn bộ mã nguồn chia thành nhiều thư mục và tệp khác nhau (ví dụ như engine, logic, renderer, interaction, utils, ...), trong đó mỗi tệp tương ứng với một **lớp (class)** hoặc một nhóm chức năng liên quan. Đây không chỉ là cách tổ chức file để dễ quản lý, mà còn thể hiện rõ nguyên lý **đóng gói (encapsulation)**.

Mỗi lớp sẽ *che giấu dữ liệu nội bộ* (thông qua private/protected) và chỉ cung cấp các *giao diện công khai* (public) để các thành phần khác tương tác. Nhờ đó, các module trong chương trình trở nên **ít phụ thuộc lẫn nhau hơn**, giúp việc bảo trì, sửa lỗi và mở rộng trở nên dễ dàng. Ví dụ, việc thay đổi cách cài đặt trong SDLRenderer sẽ không ảnh hưởng đến Engine, miễn là giao diện I_Renderer vẫn được giữ nguyên. Đây chính là nền tảng quan trọng để xây dựng các hệ thống phần mềm có cấu trúc tốt.

Đây chính là bước chuyển quan trọng trong tư duy: từ việc “làm cho chương trình chạy được” sang “thiết kế một hệ thống có thể mở rộng”. Level này không chỉ yêu cầu các bạn viết code, mà còn yêu cầu các bạn **hiểu và kiểm soát kiến trúc của chương trình**.

III.2 Cấu trúc chương trình (Program Structure)

Cấu trúc chương trình được thể hiện như sau:

```
starter-code/
├── assets/                // tài nguyên của trò chơi
│   └── ...
├── build/                // nơi compile ra bản thực thi
│   └── ...
├── src/                  // mã nguồn của trò chơi
│   ├── game/            // module game
│   │   ├── bot/         // module bot
│   │   │   └── ...
│   │   ├── interface/   // module interface
│   │   │   └── ...
│   │   └── ...
│   ├── sdl/             // module sdl
│   │   └── ...
│   ├── terminal/        // module terminal
│   │   └── ...
│   ├── utils/           // module utils hỗ trợ
│   │   └── ...
│   └── main.cpp          // file chương trình chính (.cpp)
└── CMakeLists.txt        // file CMake
```

Trong đó, cấu trúc chương trình bao gồm 4 phần chính:

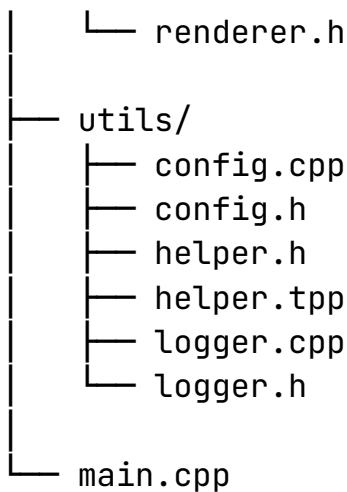
- **assets/**: nơi lưu trữ các tài nguyên (assets) của trò chơi, một số có thể để là **resources/**. Đây là thư mục lưu các tệp tin đa phương tiện (ảnh, font chữ, ...) được sử dụng để làm đồ họa cho trò chơi.
- **build/**: nơi CMake biên dịch mã nguồn, tạo các file object và xử lý liên kết thư viện **<SDL2/SDL.h>** (cùng một số thư viện khác nếu sử dụng thêm) để có thể

tiến hành tạo bản thực thi của trò chơi (game.exe).

- src/ nơi lưu trữ mã nguồn của trò chơi, những file .cpp, .h, .hpp của mã nguồn sẽ được đặt tại đây. Cấu trúc của thư mục này sẽ được đề cập chi tiết hơn ở dưới.
- CMakeLists.txt là file cấu hình CMake cho việc build bản thực thi.

Cấu trúc thư mục src/ Cấu trúc cụ thể của src/ như sau:

```
src/
├── game/
│   ├── bot/
│   │   ├── bot_fractory.cpp
│   │   ├── bot_fractory.h
│   │   ├── bot_lv1.cpp
│   │   ├── bot_lv1.h
│   │   ├── ...
│   │   ├── bot.cpp
│   │   └── bot.h
│   ├── interface/
│   │   ├── i_interaction.cpp
│   │   ├── i_interaction.h
│   │   ├── i_renderer.cpp
│   │   └── i_renderer.h
│   ├── engine.cpp
│   ├── engine.h
│   ├── logic.cpp
│   ├── logic.h
│   └── setup.h
├── sdl/
│   ├── interaction.cpp
│   ├── interaction.h
│   ├── renderer.cpp
│   └── renderer.h
└── terminal/
    ├── interaction.cpp
    ├── interaction.h
    └── renderer.cpp
```



Trong đó, bên cạnh file `main.cpp`, mã nguồn bao gồm 4 phần chính:

- Module `game/` bao gồm các file engine (GameEngine), logic (GameLogic) và setup (GameConfig). Bên cạnh đó, module này bao gồm 2 module con là `bot/` và `interface/`. Trong đó:
 - Module `bot/` định nghĩa các thuật toán của máy (bot) bao gồm 3 level là `bot_lv1`, `bot_lv2` và `bot_lv3`. Ngoài ra, ở đây có thêm bot (lớp trừu tượng) và `bot_factory` áp dụng mẫu thiết kế nhà máy (factory pattern) để tạo ra các thực thể bot theo từng `BotLevel`.
 - Module `interface/` định nghĩa hai giao diện quan trọng là `i_renderer` (GameRenderer) và `i_interface` (GameInterface). Module này là những định nghĩa về hành vi mà engine sẽ sử dụng để điều phối cùng với logic, tạo ra trò chơi hoàn chỉnh.
- Module `sdL/` bao gồm lớp cụ thể của hai giao diện được đề cập bên trên. Trong đó `renderer` sẽ định nghĩa lớp cụ thể `SDLRenderer` có trách nhiệm xử lý phần đồ họa (`SDL_Window`) của trò chơi. Bên cạnh đó, `interaction` sẽ định nghĩa lớp cụ thể `SDLInteraction` chịu trách nhiệm xử lý tương tác (`SDL_Event`). Module này sẽ sử dụng thư viện `SDL2` hoặc `SDL3` (tùy sự sáng tạo của các bạn).
- Module `terminal/` tương tự module trên, cũng bao gồm lớp cụ thể của hai giao diện. Trong đó `renderer` sẽ định nghĩa lớp cụ thể `TerminalRenderer`. Đồng thời, `interaction` định nghĩa lớp cụ thể `TerminalInteraction`. Hai lớp này có trách nhiệm tương tự như hai lớp cụ thể trên của module `sdL`; tuy nhiên không sử dụng thư viện `SDL` mà đơn giản là luồng nhập xuất cơ bản `iostream`.
- Module `utils/` bao gồm các phần hỗ trợ các bạn trong quá trình hoàn thiện mã nguồn như: `config`, `helper` (GameHelper) và `logger` (GameLogger). Các mã nguồn này được trích xuất từ mã nguồn của `level-1`.

III.3 Thành phần mã nguồn

Để tiện cho các bạn trong việc làm bài tập lớn này, hãy quan sát bảng ánh xạ từ `level-1` sang `level-2` như sau:

Bảng 1: Bảng ánh xạ các thành phần giữa Level 2 và Level 1

Thành phần Level 2	Ánh xạ	Thành phần Level 1	Ghi chú
game/			
game/engine.cpp, engine.h	→	GameEngine	Cho trước, có thể sáng tạo
game/logic.cpp, logic.h	→	GameLogic	Cần tự ánh xạ lại
game/setup.h	→	GameConfig	Cho trước, có thể sáng tạo
game/interface/			
game/interface/i_renderer.*	X		Cho trước, có thể sáng tạo
game/interface/i_interaction.*	X		Cho trước, có thể sáng tạo
game/bot/			
game/bot/bot.h, bot.cpp	X		Cho trước, có thể sáng tạo
game/bot/bot_lv1.*	→	Bot trong GameLogic	Cần tự ánh xạ lại
game/bot/...	→	...	Tương tự trên
game/bot/bot_factory.*	X		Cho trước mã giả
sdl/			
sdl/renderer.*	X		Cho trước mã giả, có thể sáng tạo, cần hoàn thiện
sdl/interaction.*	X		Cho trước mã giả, có thể sáng tạo, cần hoàn thiện
terminal/			
terminal/renderer.*	→	GameRenderer	Cần tự ánh xạ lại
terminal/interaction.*	→	GameInteraction	Cần tự ánh xạ lại
utils/			
utils/logger.*	→	GameLogger	Cho trước, có thể sáng tạo
utils/config.*	→	RunConfig và hàm parseArgs	Cho trước, có thể sáng tạo
utils/helper.*	→	GameHelper	Cho trước, có thể sáng tạo
root			
main.cpp	→	hàm main()	Cho trước, có thể sáng tạo

Từ bảng trên, có thể thấy rằng một số thành phần đã được cung cấp trong *starter code* (như Engine, Logger, Config, Interface, ...). Tuy nhiên, **các bạn không bắt buộc phải giữ nguyên cách cài đặt này** mà hoàn toàn có thể điều chỉnh, tái thiết kế hoặc mở rộng. Miễn là vẫn đảm bảo đạt được các mục tiêu đã đề ra.

Trong khi đó, các phần cần **tập trung ánh xạ lại** từ Level 1 (như Logic, Terminal Renderer, Terminal Interaction, Bot cơ bản) và đặc biệt là các **thành phần mới** (như SDL Renderer SDL Interaction, Factory, hệ thống Interface) chính là trọng tâm để các bạn hoàn thiện và thể hiện tư duy thiết kế của mình. Từ đây, phần tiếp theo sẽ trình bày cụ thể cách tiếp cận và từng bước triển khai.

IV. Hướng dẫn triển khai (Implementation Guide)

Phần này định hướng triển khai bài tập Level 2 từ *starter code*. Thay vì cung cấp lời giải cố định, hướng dẫn tập trung vào việc giúp các bạn **hiểu rõ vai trò thành phần, xác định thứ tự triển khai** và **tránh lỗi thiết kế** trong OOP & GUI.

Lộ trình bao gồm: đọc hiểu cấu trúc mã nguồn, ánh xạ thành phần từ Level 1, hoàn thiện các module (Renderer, Interaction) và tích hợp vào Engine. Các bạn nên kiểm thử từng phần trước khi hoàn thiện hệ thống.

Đây là **hướng dẫn tham khảo**. Các bạn có thể linh hoạt thay đổi cách tiếp cận, miễn là đảm bảo chương trình đáp ứng tốt các tiêu chí về **chức năng, thiết kế** và **tính mở rộng**.

IV.1 Tiền triển khai

Trước khi bước vào Level 2 với đầy đủ các khái niệm về OOP, Interface và GUI, phần này đóng vai trò như một **Level 1.5** nhằm giúp các bạn làm quen với tư duy **chia module** và **trừu tượng hoá dữ liệu** một cách rõ ràng hơn.

Nhắc lại Level 1, toàn bộ chương trình thường được viết trong một file lớn (ví dụ: `game.cpp`). Điều này giúp việc bắt đầu dễ dàng, nhưng khi chương trình phát triển, mã nguồn sẽ trở nên khó đọc, khó bảo trì và khó mở rộng. Do đó, bước tiếp theo là **tách chương trình thành nhiều thành phần độc lập**.

Quy trình thực hiện Quy trình trong phần này tương đối đơn giản:

- Bắt đầu từ file `game.cpp` của Level 1,
- Tách dần các thành phần thành các file riêng biệt, ví dụ:
 - `main.cpp` (điểm bắt đầu chương trình),
 - `config, gameConfig` (cấu hình),
 - `gameLogger` (log hệ thống),
 - `gameEngine` (điều phối),
 - `gameHelper` (hàm hỗ trợ),
 - `gameLogic` (luật chơi),
 - `gameInteraction` (input),
 - `gameRenderer` (output).

Trình tự tách có thể linh hoạt tùy theo cách tiếp cận của các bạn, tuy nhiên mục tiêu quan trọng nhất là: **mỗi thành phần phải độc lập và có trách nhiệm rõ ràng**.

Cấu trúc level-1.5 Sau khi hoàn thành, các bạn sẽ thu được một cấu trúc mã nguồn như sau:

```
level-1.5/
|
|— config.cpp
|— config.h
|— gameConfig.cpp
|— gameConfig.h
|— gameEngine.cpp
|— gameEngine.h
|— gameHelper.h
|— gameHelper.tpp
|— gameInteraction.cpp
|— gameInteraction.h
|— gameLogger.cpp
|— gameLogger.h
|— gameLogic.cpp
|— gameLogic.h
|— gameRenderer.cpp
|— gameRenderer.h
|— main.cpp
|— Makefile
```

So với phiên bản ban đầu, mã nguồn lúc này đã:

- **Ngăn gọn hơn trong từng file,**
- **Dễ đọc và dễ quản lý hơn,**
- **Tiệm cận với cách tổ chức của Level 2.**

Biên dịch nhiều file mã nguồn Tuy nhiên, việc chia nhỏ file kéo theo một vấn đề mới: **quá trình biên dịch trở nên phức tạp hơn**. Thay vì chỉ biên dịch một file duy nhất, các bạn cần:

- Biên dịch từng file .cpp thành file đối tượng .o,
- Sau đó **link** tất cả lại để tạo ra file thực thi .exe.

Để đơn giản hoá quy trình này, một file Makefile được cung cấp sẵn. Các bạn chỉ cần sử dụng:

- mingw32-make (hoặc make) để biên dịch toàn bộ chương trình,
- mingw32-make clean để xoá các file build.

Makefile mẫu cho level-1.5 Dưới đây là Makefile mẫu:

```
1 CXX = g++
2 CXXFLAGS = -std=c++20 -Wall -Wextra -O2
3
4 TARGET = game
5 SRCS = main.cpp config.cpp gameLogger.cpp gameInteraction.cpp
   gameRenderer.cpp gameLogic.cpp gameEngine.cpp
6 OBJS = $(SRCS:.cpp=.o)
7
8 all: $(TARGET)
9
10 $(TARGET): $(OBJS)
11     $(CXX) $(CXXFLAGS) -o $(TARGET) $(OBJS)
12
13 %.o: %.cpp
14     $(CXX) $(CXXFLAGS) -c $< -o $@
15
16 # detect windows
17 ifeq ($(OS),Windows_NT)
18     RM = del /Q
19     TARGET_EXT = .exe
20 else
21     RM = rm -f
22     TARGET_EXT =
23 endif
24
25 clean:
26     $(RM) *.o
27     $(RM) $(TARGET)$(TARGET_EXT)
```

Đoạn Makefile trên giúp tự động hoá toàn bộ quá trình **biên dịch (compile)** và **liên kết (link)** chương trình. Dưới đây là ý nghĩa của từng phần:

- **Trình biên dịch và cờ biên dịch:**

- CXX = g++: sử dụng trình biên dịch g++,
- CXXFLAGS: các tùy chọn như chuẩn C++20, cảnh báo lỗi (-Wall, -Wextra) và tối ưu hoá (-O2).

- **Tên chương trình và danh sách file:**

- TARGET: tên file thực thi (output),
- SRCS: danh sách các file nguồn .cpp,
- OBJS: tự động chuyển từ .cpp sang .o (object files).

- **Quy tắc build chính:**

- all: mục tiêu mặc định, sẽ build TARGET,

- \$(TARGET): \$(OBS): link tất cả file .o thành file thực thi.
- **Quy tắc biên dịch từng file:**
 - %.o: %.cpp: mỗi file .cpp được compile thành .o,
 - \$<: file nguồn đầu vào, \$@: file đầu ra.
- **Phát hiện hệ điều hành:**
 - Nếu là Windows (Windows_NT), sử dụng del,
 - Ngược lại (Linux/Mac), sử dụng rm,
 - TARGET_EXT giúp thêm đuôi .exe khi cần.
- **Dọn dẹp:**
 - make clean: xóa toàn bộ file .o và file thực thi.

Phần **Level 1.5** này đóng vai trò rất quan trọng: giúp các bạn chuyển từ tư duy “một file lớn” sang “hệ thống nhiều module độc lập”. Khi đã quen với cách tổ chức này, việc tiếp cận Level 2 (với Interface, OOP và GUI) sẽ trở nên **tự nhiên và dễ hiểu hơn rất nhiều**.

IV.2 Thiết kế luồng thực thi

Dựa trên mã nguồn *starter code*, có thể thấy rằng Engine đóng vai trò là trung tâm điều phối toàn bộ chương trình, làm việc thông qua hai interface `I_Renderer` và `I_Interaction`. Một điểm quan trọng cần nhấn mạnh: **luồng thực thi của Level 2 gần như giữ nguyên tư duy của Level 1**, chỉ khác ở chỗ các thành phần đã được *trừu tượng hoá* thông qua interface.

Luồng tuần tự Cụ thể, qua quan sát có thể thấy chương trình vẫn được tổ chức theo một **luồng tuần tự** (*sequential flow*):

`init()` → `startGame()` → `playGame()` → `endGame()` → `close()`

Trong đó:

- **init()**: khởi tạo hệ thống (Renderer và Interaction),
- **startGame()**: thực hiện các bước thiết lập trò chơi theo thứ tự:
 - hiển thị tiêu đề,
 - yêu cầu người dùng nhập size,
 - yêu cầu nhập goal,
 - chọn chế độ chơi,
 - (nếu cần) chọn bot level,
 - khởi tạo bàn cờ.

- **playGame()**: chạy vòng lặp chính của trò chơi (game loop đơn giản theo lượt),
- **endGame()**: hiển thị kết quả,
- **close()**: giải phóng tài nguyên.

Ưu điểm của luồng tuần tự Điểm đáng chú ý là toàn bộ các bước trên được xử lý theo kiểu **từng bước một, tuần tự**, ví dụ:

show title → input size → input goal → ...

Đây chính là cách tổ chức quen thuộc của Level 1, và **rất phù hợp với giao diện Terminal**:

- đơn giản,
- dễ kiểm soát luồng chương trình,
- tận dụng trực tiếp cơ chế cin/cout (buffer chung).

Vấn đề của luồng tuần tự Tuy nhiên, khi chuyển sang **giao diện đồ họa (GUI) với SDL**, cách thiết kế này bắt đầu bộc lộ hạn chế. Lý do là:

- SDL không có cơ chế **buffer đồng bộ** như istream/ostream,
- việc hiển thị và nhận input diễn ra **bất đồng bộ** (event-driven),
- UI cần được cập nhật liên tục (frame-by-frame), không thể “chờ input rồi mới vẽ” như Terminal.

Do đó, nếu giữ nguyên luồng tuần tự này, các bạn sẽ gặp một số khó khăn:

- khó xử lý sự kiện chuột/phím theo thời gian thực,
- khó thêm animation hoặc hiệu ứng,
- luồng chương trình trở nên “cứng” (static).

Sáng tạo trong thiết kế Vì vậy, trước khi triển khai chi tiết từng giao diện, các bạn **được khuyến khích suy nghĩ và thiết kế lại luồng thực thi** nếu cần. Tuy nhiên, cần đảm bảo:

- vẫn giữ **trừu tượng hoá** thông qua I_Renderer và I_Interaction,
- vẫn đảm bảo **đa hình** giữa các lớp Terminal* và SDL*,
- cố gắng tuân thủ các nguyên tắc **SOLID**.

Lưu ý rằng việc tuân thủ SOLID có thể khó trong phạm vi môn học, nên các bạn **được phép vi phạm một phần**, miễn là:

- hiểu mình đang vi phạm ở đâu,
- có thể giải thích và cải tiến trong báo cáo hoặc vấn đáp.

Luồng lặp theo trạng thái Một hướng thiết kế phổ biến và hiện đại hơn là sử dụng **game loop theo trạng thái (state-based loop)**:

```
1 while (is_running):
2     listenEvent()
3     handleEvent()    # change state
4     render()
```

Với cách tiếp cận này:

- chương trình trở nên **linh hoạt hơn**,
- dễ tích hợp GUI với FPS, animation,
- phù hợp với cách hoạt động tự nhiên của SDL.

Tóm lại, **starter code cung cấp một luồng tuần tự dễ hiểu và gần với Level 1**, nhưng nhiệm vụ của các bạn không chỉ là cài đặt lại, mà còn là **đánh giá và cải tiến thiết kế** để phù hợp hơn với các yêu cầu của một chương trình có giao diện đồ họa.

IV.3 Hoàn thiện giao diện Terminal

Đối với giao diện Terminal, mục tiêu chính là **ánh xạ lại một cách đơn giản** từ Level 1 sang cấu trúc mới sử dụng interface. Phần lớn chức năng (hiển thị bằng cout, nhập liệu bằng cin) đã được các bạn triển khai ở Level 1, nên trong Level 2 không cần đầu tư quá nhiều thời gian cho phần này. Cụ thể:

- Cài đặt TerminalRenderer và TerminalInteraction dựa trên I_Renderer và I_Interaction,
- Ánh xạ lại các hàm đã có (in menu, nhập dữ liệu, hiển thị bàn cờ, ...),
- Đảm bảo chương trình chạy đúng với luồng tuần tự đã thiết kế.

Trong bài tập này, Terminal chủ yếu đóng vai trò:

- **baseline** để kiểm tra logic chương trình,
- **môi trường hiển thị Logger** phục vụ debug.

Do đó, các bạn nên **ưu tiên hoàn thành nhanh, đúng chức năng**, tránh dành quá nhiều thời gian tối ưu phần này.

IV.4 Hoàn thiện giao diện SDL

Khác với Terminal, phần giao diện đồ họa với SDL là **trọng tâm chính** của Level 2 và cũng là phần có độ khó cao hơn đáng kể.

Nguồn học tập thư viện SDL Trước khi bắt tay vào cài đặt, các bạn nên tham khảo các nguồn học tập cơ bản:

- SDL2: [Lazy Foo's](#), [Wiki Page](#)
- SDL3: [Wiki Page](#)

Các bạn cần nắm được cấu trúc cơ bản của một chương trình SDL (khởi tạo, tạo window, render, xử lý event, huỷ tài nguyên) trước khi triển khai lớp `SDLRenderer` và `SDLInteraction`. Trong nhiều trường hợp, **việc học trước SDL rồi mới thiết kế luồng thực thi** sẽ giúp tránh được các khó khăn không cần thiết.

Nội dung trọng tâm Có hai khía cạnh chính cần tập trung:

- **Hiển thị (Rendering):**
 - Vẽ hình học cơ bản (line, rectangle, circle),
 - Hiển thị chữ (`SDL_ttf`),
 - Hiển thị hình ảnh/sprite (`SDL Texture`),
 - Chuẩn bị assets (font, hình ảnh, ...).
- **Điều khiển (Interaction):**
 - Xử lý sự kiện bàn phím (`KEY`),
 - Xử lý chuột (`MOUSE`),
 - (Nâng cao) hỗ trợ tay cầm (`JOYSTICK/CONTROLLER`),
 - Hiểu cơ chế **event loop** của SDL (lắng nghe và xử lý sự kiện).

Lưu ý: Không có yêu cầu bắt buộc về cách triển khai:

- Các bạn có thể chọn **SDL2 hoặc SDL3**,
- Có thể thiết kế UI đơn giản hoặc sáng tạo tùy ý,
- Có thể giữ luồng tuần tự hoặc chuyển sang **state-based game loop**.

Tuy nhiên, điều quan trọng nhất là:

Thiết kế luồng thực thi phải cân bằng để cả Terminal và SDL đều có thể được cài đặt thông qua cùng một lớp trừu tượng.

Nói cách khác, dù giao diện khác nhau, Engine vẫn phải hoạt động thống nhất thông qua `I_Renderer` và `I_Interaction`. Đây chính là thước đo cho một thiết kế tốt trong bài tập này.

Phần tiếp theo sẽ trình bày cho các bạn cách cài đặt môi trường phát triển (đặc biệt là SDL2/SDL3), chạy thử các ví dụ cơ bản để kiểm tra thư viện, hướng dẫn biên dịch và build mã nguồn, cũng như cách chạy chương trình với nhiều chế độ khác nhau (interactive mode, judge mode, GUI flag, verbose flag, và các tổ hợp flag).

V. Hướng dẫn thực nghiệm (Experiment Instructions)

Phần này hướng dẫn các bạn thiết lập môi trường, biên dịch và chạy thử chương trình ở nhiều chế độ khác nhau nhằm kiểm chứng tính đúng đắn của hệ thống cũng như trải nghiệm thực tế giữa các giao diện Terminal và GUI. Thông qua quá trình thực nghiệm, các bạn không chỉ đảm bảo chương trình hoạt động ổn định mà còn thu thập các minh chứng cần thiết phục vụ cho việc phân tích và trình bày trong báo cáo.

V.1 Cài đặt môi trường

Trong bài tập này, các bạn sẽ sử dụng môi trường **MSYS2 (UCRT64)** trên Windows để cài đặt công cụ biên dịch và thư viện SDL. Đây là môi trường được khuyến nghị vì đã tích hợp sẵn hệ thống quản lý gói (pacman) và hỗ trợ tốt cho cả SDL2 và SDL3.

Bước 1: Mở terminal UCRT64 và cập nhật hệ thống bằng câu lệnh:

```
1 pacman -Syu
```

Bước 2: Cài đặt công cụ build bằng câu lệnh:

```
1 pacman -S mingw-w64-ucrt-x86_64-make \  
2           mingw-w64-ucrt-x86_64-cmake \  
3           mingw-w64-ucrt-x86_64-pkg-config
```

Bước 3: Cài đặt thư viện SDL

- **SDL2 (kèm các thư viện mở rộng):**

```
1 pacman -S mingw-w64-ucrt-x86_64-SDL2 \  
2           mingw-w64-ucrt-x86_64-SDL2_image \  
3           mingw-w64-ucrt-x86_64-SDL2_ttf \  
4           mingw-w64-ucrt-x86_64-SDL2_mixer
```

- **SDL3 (phiên bản mới hơn):**

```
1 pacman -S mingw-w64-ucrt-x86_64-sdl3 \  
2           mingw-w64-ucrt-x86_64-sdl3-image \  
3           mingw-w64-ucrt-x86_64-sdl3-ttf
```

Bước 4: Kiểm tra cài đặt Sau khi cài đặt, các thư viện sẽ được lưu tại:

- Header SDL2: /ucrt64/include/SDL2/
- Header SDL3: /ucrt64/include/SDL3/

Các file thư viện (.a, .dll) sẽ nằm trong:

/ucrt64/lib/

Có thể kiểm tra nhanh bằng lệnh:

```
1 ls /ucrt64/lib/pkgconfig/sdl2.pc
```

Sau khi hoàn thành các bước trên, môi trường của các bạn đã sẵn sàng để phát triển và chạy thử chương trình với cả **SDL2** và **SDL3**. Việc lựa chọn phiên bản nào phụ thuộc vào nhu cầu và mức độ quen thuộc của các bạn với từng thư viện.

V.2 Kiểm tra cài đặt môi trường

Sau khi hoàn tất quá trình cài đặt, bước tiếp theo là **kiểm tra lại môi trường** để đảm bảo tất cả công cụ và thư viện đã hoạt động đúng. Phần này giúp các bạn tránh các lỗi phổ biến trước khi bắt đầu triển khai bài tập chính.

Kiểm tra công cụ build

- Kiểm tra make:

```
1 mingw32-make --version
```

- Kiểm tra cmake:

```
1 cmake --version
```

- Kiểm tra trình biên dịch g++:

```
1 g++ --version
```

Kiểm tra thư viện SDL

- Kiểm tra SDL2:

```
1 pkg-config --modversion sdl2
```

- Kiểm tra SDL3:

```
1 pkg-config --modversion sdl3
```

Nếu các lệnh trên trả về version (ví dụ: 2.x.x hoặc 3.x.x) thì thư viện đã được cài đặt thành công.

Kiểm tra bằng chương trình mẫu Các bạn có thể sử dụng mã nguồn tại thư mục:

[test-sdl/](#)

Tải mã nguồn này về để kiểm tra tính sẵn sàng của thư viện SDL2 và SDL3 trên máy tính cá nhân. Thư mục mã nguồn bao gồm:

- test-sdl2.cpp: chương trình thử nghiệm SDL2,
- test-sdl3.cpp: chương trình thử nghiệm SDL3,
- Makefile: hỗ trợ build tự động.

Sử dụng các lệnh sau trong terminal:

```
1 mingw32-make sdl2    # build version SDL2
2 mingw32-make sdl3    # build version SDL3
```

Sau khi build thành công, chạy chương trình:

```
1 ./test-sdl2
2 ./test-sdl3
```

Nếu cửa sổ SDL hiển thị đúng, môi trường đã sẵn sàng.

Dọn dẹp file build

```
1 mingw32-make clean
```

/medskip

Bước kiểm tra này giúp đảm bảo rằng toàn bộ hệ thống (compiler, build tool, thư viện SDL) hoạt động ổn định trước khi các bạn bắt đầu tích hợp vào dự án chính.

V.3 Biên dịch chương trình

Sau khi hoàn tất việc cài đặt môi trường và kiểm tra thư viện, bước tiếp theo là **biên dịch chương trình**. Ở đây, Level 1.5 và Level 2 sử dụng hai cách tiếp cận khác nhau để giúp các bạn hiểu rõ hơn về hệ thống build.

Biên dịch Level 1.5 (sử dụng Makefile) Ở Level 1.5, các bạn đã làm quen với việc sử dụng Makefile để tự động hoá quá trình biên dịch và liên kết:

```
1 mingw32-make
```

Lệnh này sẽ:

- Biên dịch từng file .cpp thành file .o,
- Sau đó liên kết (link) các file .o lại thành file thực thi .exe.

Để dọn dẹp:

```
1 mingw32-make clean
```

Cách làm này giúp các bạn hiểu rõ quy trình build ở mức thấp (compile → link), nhưng khi dự án lớn dần, việc quản lý sẽ trở nên phức tạp.

Biên dịch Level 2 (sử dụng CMake) Ở Level 2, chúng ta chuyển sang sử dụng CMake – một công cụ build hiện đại giúp tự động hoá toàn bộ quá trình cấu hình và biên dịch.

Các bước thực hiện:

```
1 # 1. Create build folder (pass if created)
2 mkdir build
3 cd build
4
5 # 2. Config project
6 cmake ..
7
8 # 3. Compile
9 cmake --build .
```

CMake làm gì khi chạy các lệnh trên?

- `cmake ..`:
 - Đọc file `CMakeLists.txt`,
 - Tự động tìm tất cả file `.cpp` trong thư mục `src/`,
 - Tìm và cấu hình thư viện SDL (SDL2, SDL2_ttf),
 - Sinh ra hệ thống build phù hợp với môi trường (Makefile hoặc Ninja).
- `cmake --build .`:
 - Thực hiện biên dịch toàn bộ project,
 - Tự động link các thư viện SDL cần thiết,
 - Tạo file thực thi `game.exe`.

Một số điểm đáng chú ý trong `CMakeLists.txt`:

- `file(GLOB_RECURSE ...)`: tự động lấy toàn bộ source trong `src/`,
- `find_package(...)`: tìm thư viện SDL đã cài trong hệ thống,
- `target_link_libraries(...)`: liên kết SDL vào chương trình,
- `target_include_directories(...)`: thiết lập đường dẫn header,
- `add_custom_command`: tự động copy thư mục `assets/` sau khi build.

So với Makefile, CMake giúp các bạn **trừu tượng hoá quá trình build**, giảm thiểu lỗi cấu hình và dễ dàng mở rộng khi dự án trở nên phức tạp hơn (đa thư viện, đa nền tảng).

Lưu ý về việc sử dụng SDL2 / SDL3 trong CMake Mặc định, `CMakeLists.txt` đã được cấu hình sẵn để sử dụng **SDL2** và **SDL2_ttf**. Nếu các bạn giữ nguyên lựa chọn này thì không cần thay đổi gì thêm.

Tuy nhiên, trong trường hợp:

- Muốn **chuyển sang SDL3**, hoặc
- Muốn **bổ sung thêm các thư viện SDL khác** (ví dụ: SDL2_image, SDL2_mixer), các bạn cần chỉnh sửa đồng bộ ở **3 vị trí chính** trong CMakeLists.txt:

1. Tìm package:

```
1 # SDL2 (default)
2 find_package(SDL2 REQUIRED)
3 find_package(SDL2_ttf REQUIRED)
4
5 # SDL3 (if needed)
6 # find_package(SDL3 REQUIRED)
```

2. Include directories:

```
1 target_include_directories(game PRIVATE
2     ${SDL2_INCLUDE_DIRS}
3     # ${SDL3_INCLUDE_DIRS}
4 )
```

3. Link libraries:

```
1 target_link_libraries(game PRIVATE
2     mingw32
3     SDL2::SDL2main
4     SDL2::SDL2
5     SDL2_ttf::SDL2_ttf
6     # SDL3::SDL3
7 )
```

Nguyên tắc quan trọng: Khi thay đổi phiên bản SDL, các bạn phải đảm bảo **tính nhất quán** giữa:

- find_package,
- include_directories,
- link_libraries,
- và cả #include trong mã nguồn (SDL2/SDL.h vs SDL3/SDL3.h).

Việc không đồng bộ giữa các phần này là một trong những nguyên nhân phổ biến dẫn đến lỗi biên dịch hoặc lỗi liên kết (linking errors).

Sau bước này, các bạn đã có thể biên dịch và chạy chương trình Level 2 một cách ổn định, sẵn sàng cho việc thử nghiệm các chế độ chạy khác nhau ở phần tiếp theo.

V.4 Chạy chương trình

Sau khi biên dịch thành công bằng lệnh:

```
1 cmake --build .
```

bên trong thư mục build/ sẽ xuất hiện file thực thi game.exe. Các bạn có thể chạy chương trình với nhiều chế độ khác nhau thông qua các **flag** dòng lệnh.

Chạy chế độ tương tác (interactive mode) có cú pháp như sau:

```
1 ./game.exe
```

- Đây là chế độ mặc định (không cần flag),
- Chương trình chạy ở chế độ **tương tác**,
- Giao diện mặc định là **Terminal**,
- Logger mặc định ghi vào file log.txt.

Chạy chế độ chấm điểm (judge mode) có cú pháp như sau:

```
1 ./game.exe -j
2 # or
3 ./game.exe --judge
```

- Chế độ **không tương tác**,
- Giao diện luôn là **Terminal**,
- Luồng vào (istream) được điều hướng từ file.

Thường đi kèm với:

```
1 ./game.exe -j -i ../testcase/input1.txt
```

- -i / --input: chỉ định file input,
- Được sử dụng trong hệ thống grader.py,
- Có thể dùng để test từng testcase riêng lẻ.

Chạy chế độ đồ họa (graphic flag) có cú pháp như sau:

```
1 ./game.exe -g
2 # or
3 ./game.exe --gui_flag
```

- Bật chế độ **đồ họa**,
- Giao diện chuyển sang **SDL (GUI)**,
- Renderer và Interaction sẽ được thay bằng phiên bản SDL.

Chạy chế độ gỡ lỗi (verbose flag) có cú pháp như sau:

```
1 ./game.exe -v
2 # or
3 ./game.exe --verbose
```

- Bật chế độ **debug**,
- Logger chuyển từ mức INFO (mặc định) xuống DEBUG,
- Hiển thị thêm thông tin chi tiết trong quá trình chạy.

Chạy với điều hướng logger có cú pháp như sau:

```
1 ./game.exe -l log_output.txt
```

- -l / --log: chỉ định file log,
- Mặc định là log.txt,
- Có thể đổi sang file khác để tiện theo dõi.

Ngoài ra, nếu để trống đường dẫn:

```
1 ./game.exe -l
```

- Logger sẽ in trực tiếp ra **Terminal**,
- Thường dùng khi debug kết hợp với -v,
- Nên đặt flag này ở cuối để dễ quan sát output.

Chạy các tập hợp chế độ Các flag có thể kết hợp linh hoạt:

```
1 ./game.exe -j -i ../testcase/input1.txt
2 ./game.exe -v -g -l
```

- -j + -i: chạy chấm điểm với input file,
- -v + -g + -l: chạy GUI + debug + log ra terminal.

V.5 Chạy mã chấm điểm (Grader)

Hệ thống chấm điểm tự động được cung cấp dưới dạng script Python:

```
1 python grader.py --target build/game
```

- --target: đường dẫn tới file thực thi (không cần .exe),
- --testcase_dir: (tùy chọn) chỉ định thư mục chứa testcase.

Công cụ này sẽ tự động chạy nhiều test case và kiểm tra kết quả, giúp các bạn đánh giá nhanh tính đúng đắn của chương trình.

V.6 Ghi nhận kết quả vào báo cáo

Ở Level 2, trọng tâm không còn nằm ở việc kiểm thử logic thuần túy, mà chuyển sang **thiết kế hệ thống, giao diện người dùng và cách tổ chức chương trình**. Do đó, các minh chứng trong báo cáo cũng sẽ thay đổi tương ứng.

Các bạn cần tổng hợp các nội dung sau vào thư mục:

./results/

1. **Ảnh chụp màn hình (Terminal vs Graphic)** Chụp lại các màn hình tương tác chính của trò chơi ở **cả hai giao diện** (Terminal và SDL).

- Mỗi giao diện nên có các ảnh tương ứng (menu, board, player move, result, ...),
- Đặt tên file theo cặp để dễ so sánh với nhau, ví dụ: terminal_menu.png – sdl_menu.png, terminal_board.png – sdl_board.png, ...

Mục tiêu là thể hiện rõ **sự tương ứng giữa hai cách cài đặt** dựa trên cùng một interface.

2. **Tài nguyên game (Assets)** Nếu chương trình sử dụng tài nguyên đồ họa, các bạn cần mô tả và minh chứng rõ ràng:

- **Font chữ:** tên font (family), kiểu (style), kích thước sử dụng,
- **Hình ảnh / sprite:** file ảnh gốc, cách load ảnh vào chương trình, cách cắt (extract frame) nếu có animation,
- Các tài nguyên khác (âm thanh, icon, ...) nếu có.

Bắt buộc phải có **ít nhất một hình minh họa pipeline:**

Assets (ảnh/font) → Render → Màn hình cuối cùng

3. **File Log** Yêu cầu tương tự Level 1, nhưng tập trung vào:

- Các dòng log mức DEBUG,
- Luồng thực thi của Engine (init, start, play, end),
- Các sự kiện quan trọng (input, render, bot, ...).

Nếu sử dụng SDL một cách đầy đủ, file log chi tiết sẽ là minh chứng quan trọng cho việc:

- Hiểu rõ sự tách biệt giữa **UI (User Interface)** và **Dev Tool (Logger)**,
- Theo dõi và debug các hành vi phức tạp trong giao diện đồ họa.

Không giống Level 1, báo cáo từ grader không còn là yêu cầu bắt buộc. Thay vào đó, các minh chứng về **thiết kế, tổ chức mã nguồn và trải nghiệm giao diện** sẽ là cơ sở chính để đánh giá bài làm của các bạn.

Lưu ý: Mọi kết quả trong báo cáo cần được trình bày trung thực, khớp với dữ liệu log và ảnh chụp màn hình thực tế từ máy tính cá nhân. Các trường hợp sai lệch dữ liệu sẽ bị coi là không hoàn thành yêu cầu thực nghiệm.

VI. Yêu cầu báo cáo (Report Requirements)

Báo cáo là phần thể hiện rõ nhất mức độ hiểu và khả năng thiết kế của các bạn trong bài tập Level 2. Không chỉ dừng lại ở việc chương trình chạy được, báo cáo cần trình bày cách các bạn tổ chức mã nguồn, xây dựng luồng thực thi và triển khai giao diện (Terminal và GUI) một cách hợp lý. Nội dung cần ngắn gọn, có minh chứng rõ ràng và phản ánh trung thực quá trình thực hiện.

VI.1 Định dạng và Quy định chung

- **Hình thức:** Nộp file định dạng **PDF**. Giữ nguyên cấu trúc nội dung.
- **Công cụ:** Khuyến khích sử dụng $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ (Overleaf) hoặc MS Word/Google Docs nhưng phải xuất PDF.
- **Yêu cầu:** Nội dung báo cáo phải phản ánh chính xác mã nguồn đã nộp. Mọi hành vi sao chép sẽ bị xử lý theo quy định.
- **Ngôn ngữ:** Tiếng Việt, trích dẫn thuật ngữ Tiếng Anh (lần đầu nhắc đến).
- **Đặt tên:** lưu file báo cáo với định dạng tên sau:

`report_level-2_<HoVaTen>_<mssv>.pdf`

Ví dụ: `report_level-2_NguyenVanA_2502XXXX.pdf`

VI.2 Cấu trúc nội dung (Outline)

Báo cáo cần tuân thủ đúng 06 mục sau đây:

1. Tổng quan (Overview)

- Thông tin cá nhân (Họ tên, MSSV).
- Tổng quan thiết kế luồng thực thi được áp dụng trong bài làm.
- Giao diện đồ họa sử dụng hình thức hiển thị nào (font chữ, vẽ hình học, vẽ sprite ảnh, ...) và hỗ trợ kiểu tương tác nào (chuột, bàn phím, tay cầm console, ...).

2. Chi tiết cài đặt (Implementation Details)

Phần này tập trung trả lời các vấn đề kỹ thuật cốt lõi. Các bạn không cần trình bày toàn bộ mã nguồn mà tập trung vào:

- **Tư duy thiết kế:** Trình bày chi tiết cách xây dựng luồng thực thi của Engine, có thay đổi gì so với luồng ban đầu? Tại sao lại lựa chọn thiết kế này? Thiết kế này có ưu điểm và nhược điểm gì?
- **Trả lời câu hỏi:** Trả lời các câu hỏi trọng tâm về luồng thiết kế và mức độ đạt các mục tiêu được trình bày ở bên dưới.
- **Giao diện đồ hoạ:** Trình bày chi tiết về xử lý giao diện đồ hoạ bao gồm 2 phần:
 - **Hiển thị:** Trình bày về hình thức hiển thị trên giao diện GUI. Tập trung vào triết lý/cơ sở cài đặt. Lý do lựa chọn hình thức này? Hiệu năng hiển thị đồ hoạ? (Có thể dùng kết quả log để tính thời gian render hình ảnh, nếu làm FPS có thể tính trung bình số frame, thời gian xử lý một frame, ...).
 - **Tương tác:** Trình bày về cách mà trò chơi xử lý tương tác của người dùng. Tập trung vào cách cài đặt. Chương trình hỗ trợ những tương tác nào? Đây là khó khăn lớn nhất trong quá trình cài đặt tương tác?

Câu hỏi yêu cầu:

1. **Lập trình hướng đối tượng:** Trò chơi của các bạn có thể hiện đầy đủ 4 nguyên lý của lập trình hướng đối tượng (OOP) không? Bên cạnh những gì mã nguồn chính đã cung cấp, các bạn có cài đặt thêm hoặc sáng tạo thay thế để làm rõ nguyên lý OOP? Chỉ ra minh chứng cho từng nguyên lý.
2. **Thiết kế cân bằng hai giao diện:** Thiết kế của bạn có cân bằng giữa việc xử lý giao diện Terminal và giao diện GUI không? Nếu có, đâu là thứ cốt lõi giúp cho thiết kế của bạn hài hoà? Nếu không, đâu là vấn đề mà bạn cảm thấy khó khăn trong quá trình thiết kế?
3. **Đánh giá mức độ tuân thủ SOLID:** Thiết kế của bạn có tuân thủ các nguyên tắc SOLID không? Chỉ rõ từng nguyên tắc đã tuân thủ với minh chứng cụ thể. Trong đó, đối với các nguyên tắc chưa tuân thủ (có vi phạm một phần), chỉ ra phần vi phạm và diễn giải.
4. **Sử dụng mẫu thiết kế:** Chương trình của các bạn có sử dụng thêm bất kỳ mẫu thiết kế (Design Pattern) nào không? Nếu có, trình bày chi tiết mẫu thiết kế và minh chứng cài đặt.

3. Kết quả thực nghiệm (Experiment Results)

Cung cấp minh chứng trực quan về sự vận hành của chương trình:

- **Ảnh chụp màn hình (Terminal vs Graphic):** So sánh giao diện các màn hình tương tác chính của trò chơi ở cả hai giao diện (Terminal và SDL) (*sử dụng ảnh minh chứng chụp ở phần trước*). Đưa ra nhận xét cá nhân.
- **Tài nguyên game (Assets):** Mô tả cụ thể kèm hình ảnh minh chứng (*sử dụng ảnh minh chứng chụp ở phần trước*) cho các tài nguyên được sử dụng trong trò chơi.

Nếu cài tài nguyên này được lấy từ một nguồn cụ thể, ghi rõ nguồn. Nếu các bạn tự sáng tạo ra các tài nguyên này, hãy nói thêm về quy trình sáng tạo đa phương tiện.

Lưu ý: các bạn phải có ít nhất **một hình ảnh minh họa quá trình** từ tài nguyên raw (assets raw) sau khi render (tổng hợp) và hiển thị ra màn hình của trò chơi; để có cái nhìn hình dung cụ thể hơn.

- **Trích dẫn Log:** Trích dẫn các đoạn log tiêu biểu từ quá trình chạy thực nghiệm. Tập trung vào việc nắm được sự khác nhau giữa giao diện UI và DevTool. Đoạn log nên thể hiện được thiết kế của luồng thực thi Engine, quá trình xử lý các sự kiện quan trọng: hiển thị (render) và tương tác (interact).

4. Nhận xét và tổng kết bài làm

Phần này là không gian để bản thân tự đúc kết kinh nghiệm và nhìn nhận lại quá trình thực hiện dự án. Hãy trả lời ngắn gọn các câu hỏi gợi ý sau:

- **Về phong cách lập trình:** Thông qua bài tập này, việc sử dụng phong cách lập trình hướng đối tượng (Object-Oriented Programming) có điểm mạnh và điểm yếu gì trong việc thiết kế luồng trò chơi? Cách tiếp cận này đối với một dự án lớn - ví dụ như trò chơi này khi trở nên phức tạp hơn có dễ bảo trì hơn? Theo bạn, có phải dự án nào cũng phù hợp với phong cách lập trình này?
- **Về giao diện người dùng:** Khi được tận tay phát triển một phần mềm có giao diện đồ họa (GUI) như này, bạn thấy những phần nào thú vị và gặp phải những khó khăn nào trong quá trình làm? So với Terminal Interface thì bạn thích giao diện nào hơn? Tại sao?
- **Về tổ chức mã nguồn:** Ở bài tập này, mã nguồn đã được tách ra và quản lý ở nhiều file (*multiple files*), mỗi file cũng được ghi rõ các chú thích kỹ thuật riêng (*docstring/comment*); điều này có hiệu quả cho việc phát triển lâu dài? So sánh ưu/nhược điểm với cách tiếp cận tập trung mã nguồn vào một file? Khi nào nên lựa chọn tổ chức mã nguồn theo cách cách này?
- **Về thiết kế chương trình:** Bài tập này cũng đã mang lại cho các bạn một phần nhỏ của việc thiết kế chương trình. Bên cạnh việc lập trình mã nguồn (*coding*), thứ mà các công cụ AI (cụ thể là các mô hình *LLM*) bây giờ đã có thể làm rất tốt, thì vẫn còn một thứ quan trọng trước khi một chương trình được cụ thể hoá - đó là thiết kế. Sau phần làm này, các bạn nhìn nhận vai trò của con người và công cụ AI thế nào trong một quy trình phát triển phần mềm? Liệu con người có thể bị thay thế? Vấn đề của các công cụ AI hiện tại là gì?
- **Bài học rút ra:** Khó khăn lớn nhất mà bạn đã gặp và vượt qua được là gì? Đó là vấn đề thiết kế hay lập trình?

5. Ghi chú sử dụng AI (AI Disclosure)

Sinh viên cần trung thực khai báo việc sử dụng các công cụ AI (như ChatGPT, GitHub Copilot,...) trong quá trình làm bài:

- Mục đích sử dụng (Ví dụ: Giải thích lỗi, gợi ý cấu trúc code, sửa lỗi LaTeX).
- Mức độ đóng góp của AI vào sản phẩm cuối cùng.

Đọc kỹ phần **Chính sách sử dụng AI**, để nắm được các lưu ý quan trọng trước khi tiến hành sử dụng AI trong quá trình làm bài.

6. Nguồn tham khảo (References)

Liệt kê các tài liệu, trang web hoặc thư viện (nếu có) đã sử dụng trong quá trình tìm hiểu và thực hiện bài tập.

Báo cáo cần được trình bày rõ ràng, súc tích. Các hình ảnh minh họa phải sắc nét và có chú thích đầy đủ. Đây là cơ sở quan trọng để giảng viên đánh giá mức độ hiểu bài và kỹ năng triển khai thực tế của các bạn.

VII. Chính sách sử dụng công cụ AI (AI Usage Policy)

Trong khóa học này, các bạn **được phép sử dụng các công cụ AI** (ví dụ: ChatGPT, Copilot, Claude, Gemini, v.v.) như một công cụ hỗ trợ học tập. Tuy nhiên, việc sử dụng AI cần tuân thủ các nguyên tắc sau.

VII.1 Nguyên tắc chung

- AI chỉ nên được sử dụng như **công cụ hỗ trợ học tập**, không phải để thay thế hoàn toàn quá trình suy nghĩ của các bạn.
- Sinh viên cần **hiểu rõ mã nguồn** mà mình nộp.
- Sinh viên phải có khả năng **giải thích cách hoạt động của chương trình**.

Trong quá trình chấm bài hoặc vấn đáp, nếu các bạn không thể giải thích mã nguồn của mình, bài làm có thể bị đánh giá lại.

VII.2 Yêu cầu khai báo

Nếu các bạn sử dụng AI trong quá trình làm bài, cần khai báo rõ trong báo cáo. Việc khai báo này giúp đảm bảo tính minh bạch trong học tập.

Thông tin khai báo nên bao gồm:

- Tên công cụ AI đã sử dụng
- Mục đích sử dụng (nên viết chi tiết)
- Mức độ sử dụng (ước lượng)

Ví dụ:

AI Tool: ChatGPT

Purpose: gợi ý ý tưởng cho hàm checkWin()

Estimated usage: ~20%

VII.3 Những việc không được phép

Sinh viên **không được phép**:

- Sao chép toàn bộ mã nguồn do AI sinh ra mà không hiểu nội dung
- Nộp bài giống nhau giữa các nhóm
- Sử dụng AI để vượt qua hệ thống kiểm tra mà không thực sự làm bài

Nếu phát hiện hành vi vi phạm học thuật, bài làm có thể bị xử lý theo quy định của môn học và của nhà trường.

VII.4 Mục tiêu của chính sách

Mục tiêu của chính sách này không phải là cấm AI, mà là khuyến khích các bạn:

- sử dụng AI một cách có trách nhiệm
- hiểu rõ mã nguồn của mình
- phát triển kỹ năng lập trình và tư duy giải quyết vấn đề

Đội ngũ trợ giảng thực hành môn học chúng mình khuyến khích các bạn làm bài tập này bằng đúng khả năng của bản thân. Dù kết quả có như thế nào thì đây cũng là một trải nghiệm đáng được dành thời gian và tâm sức của mình vào.

Công nghệ AI là một bước tiến lớn giúp thay đổi cách chúng ta học lập trình, nhưng kỹ năng tư duy logic và khả năng làm chủ mã nguồn mới là giá trị cốt lõi đi cùng các bạn trong sự nghiệp. Hãy coi AI như một người cộng sự thông minh để trao đổi ý tưởng, chứ đừng biến mình thành "người vận chuyển" mã nguồn từ máy tính vào bài tập.

Chúc các bạn có một hành trình trải nghiệm thú vị, học hỏi được nhiều điều mới và tự hào về sản phẩm do chính tay mình hoàn thiện!

VIII. Nộp bài (Submission)

Sau đây là hướng dẫn chuẩn bị cho phần nộp bài. Yêu cầu các bạn đọc kỹ để nộp bài đúng yêu cầu.

VIII.1 Các thành phần cần nộp

Bài nộp cần bao gồm các thành phần sau:

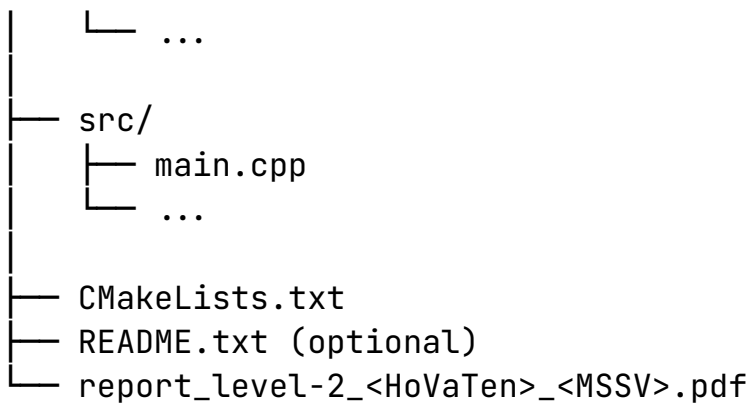
- **Mã nguồn chương trình** (các file .cpp)
- **Tài nguyên trò chơi** (các file đa phương tiện .png, .ttf, .jpg, ...)
- **Báo cáo PDF** (file .pdf)
- **Thư mục kết quả thực nghiệm** (folder results/)

Toàn bộ các file cần được đặt trong một thư mục duy nhất trước khi nén.

VIII.2 Cấu trúc thư mục

Các bạn nên tổ chức thư mục nộp bài theo cấu trúc sau:

```
proeject_level_2/
├── assets/
│   ├── menu.png
│   ├── font.ttf
│   └── ...
├── results/
│   ├── sdl-menu.png
│   ├── terminal-menu.png
│   ├── sdl-display_board.png
│   ├── ...
│   ├── sdl-player_move.png
│   ├── ...
│   ├── sdl-bot_move.png
│   ├── ...
│   └── log.txt
├── logs/
│   ├── log_graphic_flag.txt
│   └── log_default.txt
```



Trong đó:

- report_level-2_<HoVaTen>_<MSSV>.pdf là file báo cáo PDF
- src/ chứa toàn bộ mã nguồn của bài tập lớn sau khi được hoàn thiện
- assets/ chứa toàn bộ tài nguyên của trò chơi sau khi được hoàn thiện
- logs/ chứa file log được tạo ra khi chạy chương trình (có thể là phần log luồng chạy giao diện đồ họa, ...); *phần log được dùng trong báo cáo nên được để vào thư mục này (minh chứng)*
- results/ chứa các ảnh chụp màn hình trong quá trình tương tác giao diện game, so sánh tương quan giữa giao diện GUI và Terminal; *các ảnh dùng trong báo cáo nên được để vào thư mục này (minh chứng)*
- Một số file hỗ trợ như CMakeLists.txt, ...các file cần sử dụng trong quá trình biên dịch chương trình (nếu cần thiết)
- (tùy chọn) README.txt mô tả ngắn gọn cách biên dịch và chạy chương trình

VIII.3 Định dạng nộp bài

Thư mục bài làm cần được nén thành một file:

project_level-2_<HoVaTen>_<mssv>.zip

Ví dụ:

proeject_level-2_NguyenVanA_2502XXXX.zip

Chỉ nộp **một file nén duy nhất**.

VIII.4 Thời hạn nộp bài

Thời hạn nộp bài sẽ được thông báo trên hệ thống của môn học.

Các bạn nên nộp bài trước hạn để tránh các vấn đề kỹ thuật vào thời điểm gần deadline.

VIII.5 Lưu ý

- Đảm bảo chương trình có thể biên dịch và chạy được, trong trường hợp cần sử dụng thêm hoặc có cách biên dịch khác, cần bổ sung đầy đủ file hỗ trợ và hướng dẫn biên dịch;
- Kiểm tra lại file nén trước khi nộp;
- Không nộp các file không cần thiết (file object, file tạm, v.v.) và không cần nộp thư mục build.